

Architecting Discovery: A Blueprint for a World-Class E-Commerce Browsing Experience

A System Design for Serving 1 Million+ Daily Active Users at <100ms Latency.



Our System Delivers Three Tiers of Relevance



Popularity (The Foundation)

Identifies products consistently loved over time. Computed daily via offline batch processing.



Trending (The Pulse)

Detects what's 'hot' right now. Insights are generated in under 5 minutes via real-time stream processing.

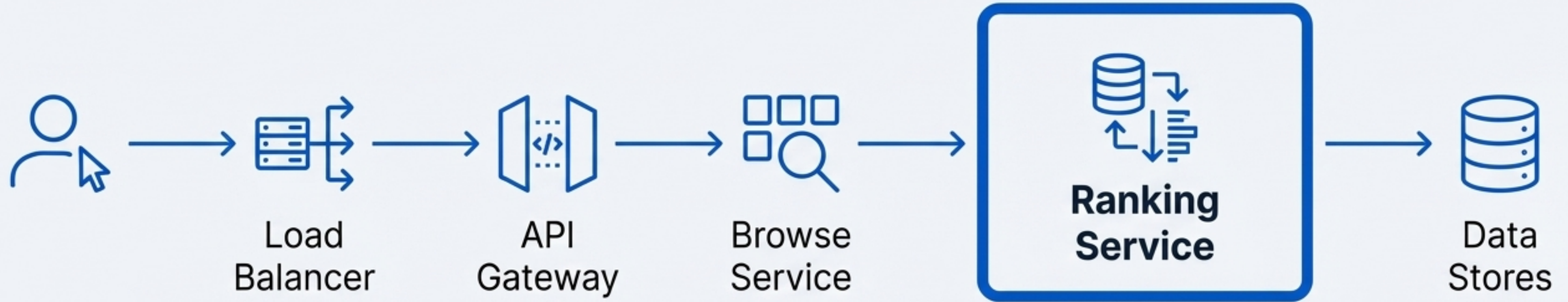


Personalised (The Reflection)

Tailors rankings to be most relevant to *you*, based on your unique behaviour.

The goal is to blend these three signals seamlessly, creating the perfect product ranking for every user, every time.

The 100ms Sprint: From User Click to Rendered Page



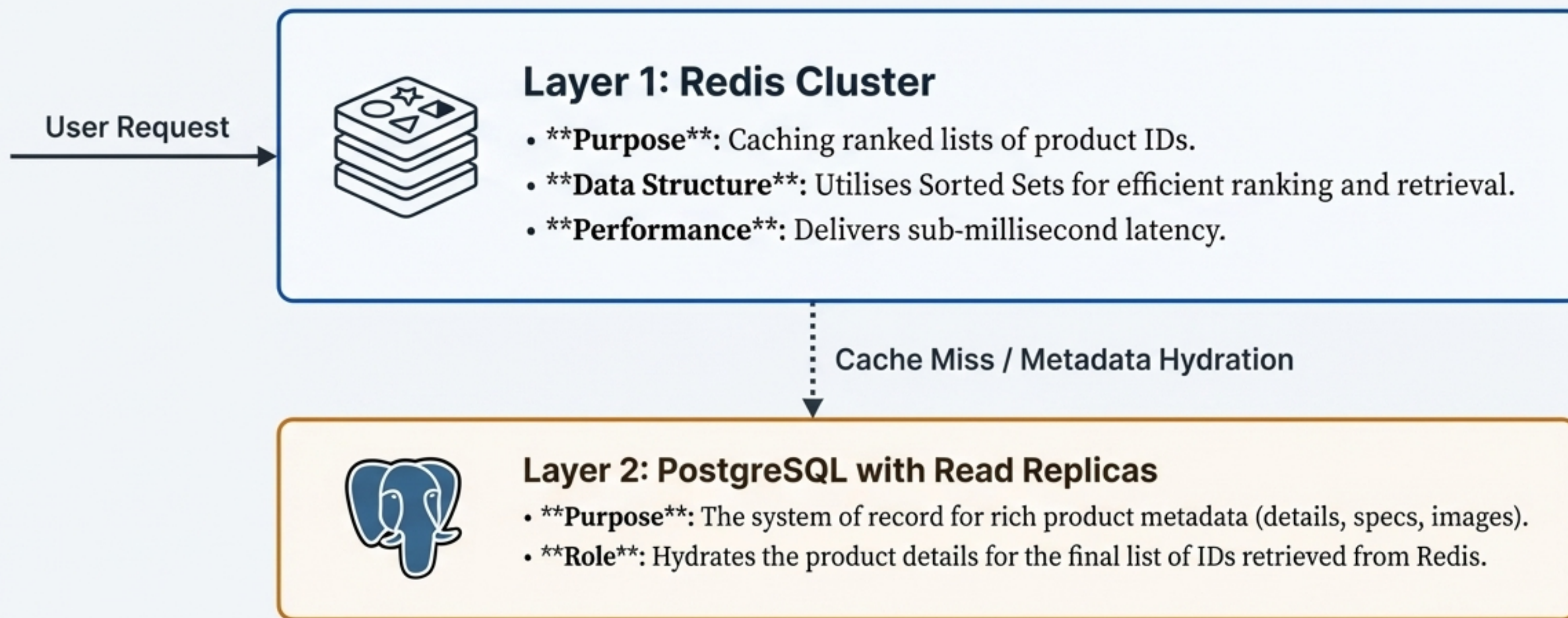
API Gateway: Handles rate limiting and authentication.

Browse Service: Manages product fetching, filtering, and pagination logic.

Ranking Service: The core intelligence hub that fetches candidate lists (Popular, Hot, Personalised) and merges them into a final ranked list for the user.

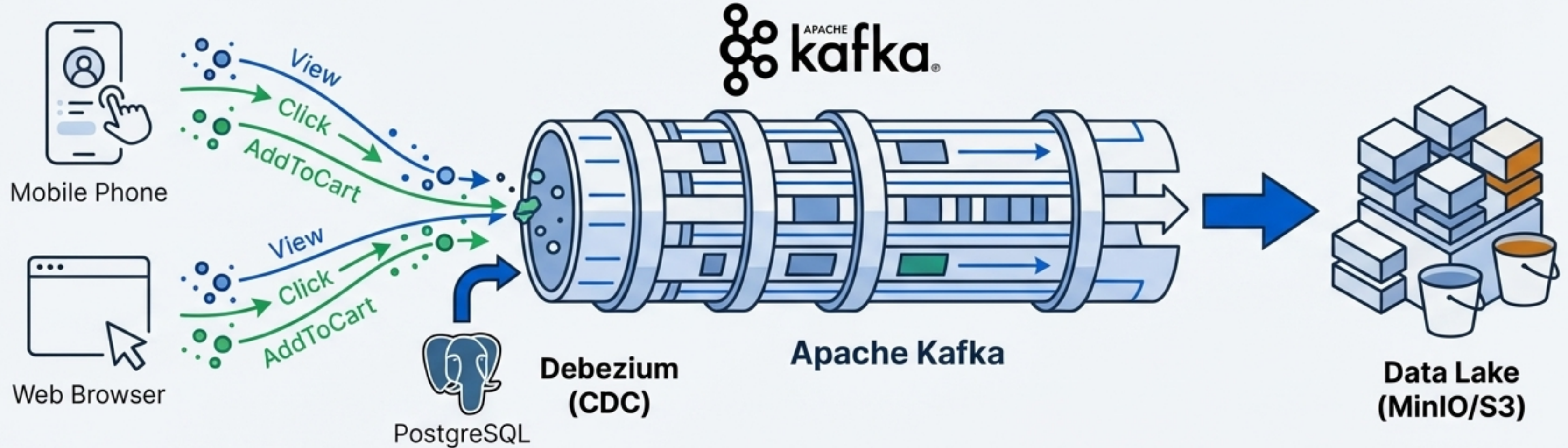
The read path is aggressively optimised for sub-millisecond data retrieval from pre-computed views and caches. The database is a fallback, not the primary source.

A Read-Optimised Data Stack for Serving at Speed



This separation of concerns—IDs in Redis, metadata in PostgreSQL—is critical for performance, shielding the primary database from high-volume read traffic.

The Source of Truth: Ingesting Millions of User Events

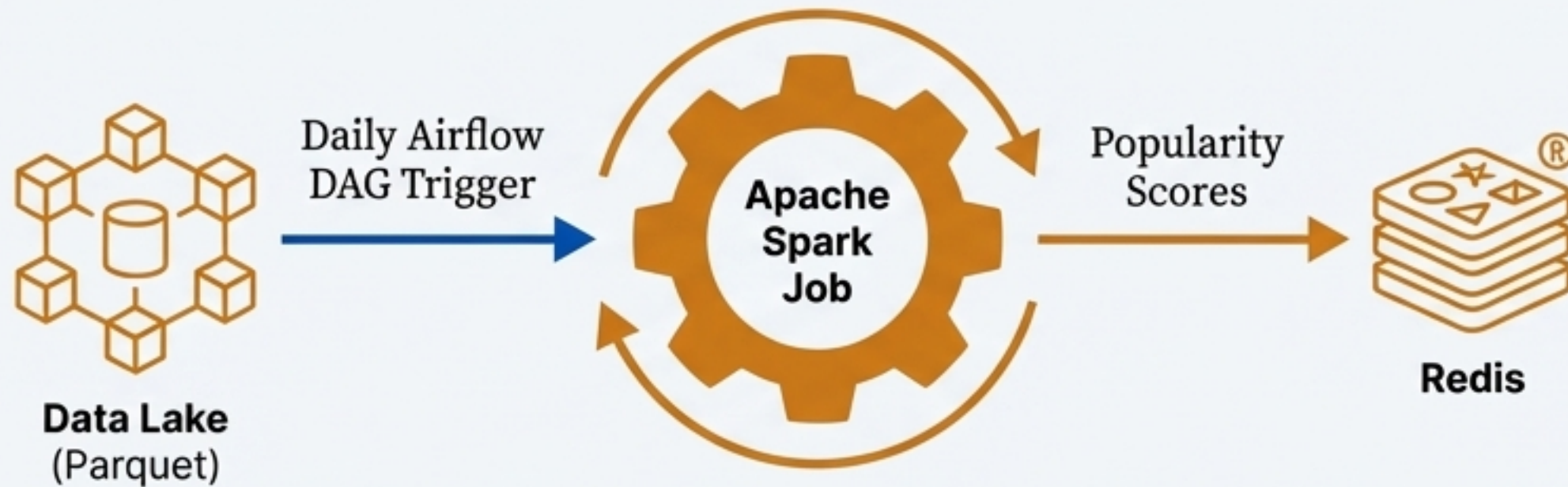


Key Technologies

-  **Apache Kafka:** Provides a durable, replayable log for all user behaviour and database changes.
-  **Debezium:** Captures change data capture (CDC) events from PostgreSQL.
-  **Parquet Format:** Used for efficient columnar storage in the data lake, partitioned by event type and date for optimised analytics.

Key Insight: Kafka acts as a durable, replayable system backbone, decoupling event producers from the various processing pipelines.

The Foundation: Calculating Daily Popularity with Apache Spark

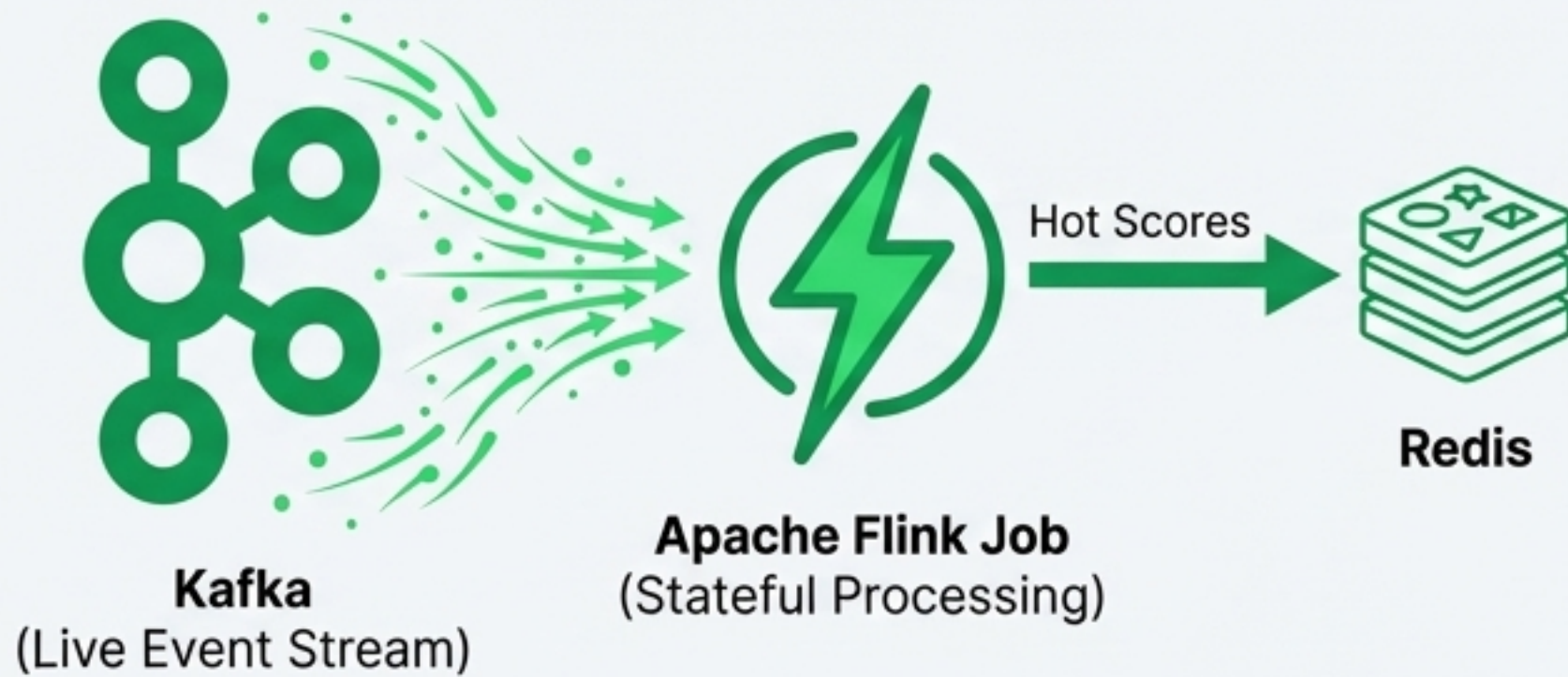


Process Explained

1. An Airflow DAG triggers a daily Spark job.
2. The job reads the last 24 hours of user engagement events from the data lake.
3. A time-decay algorithm is applied to weigh recent events more heavily.
4. New popularity scores are computed for all products.
5. The final ranked lists are written to Redis Sorted Sets for fast retrieval.

Batch processing is the most cost-effective method for analysing large historical datasets where a 24-hour data freshness is acceptable.

The Pulse: Detecting Hot Items with Apache Flink



Process Explained

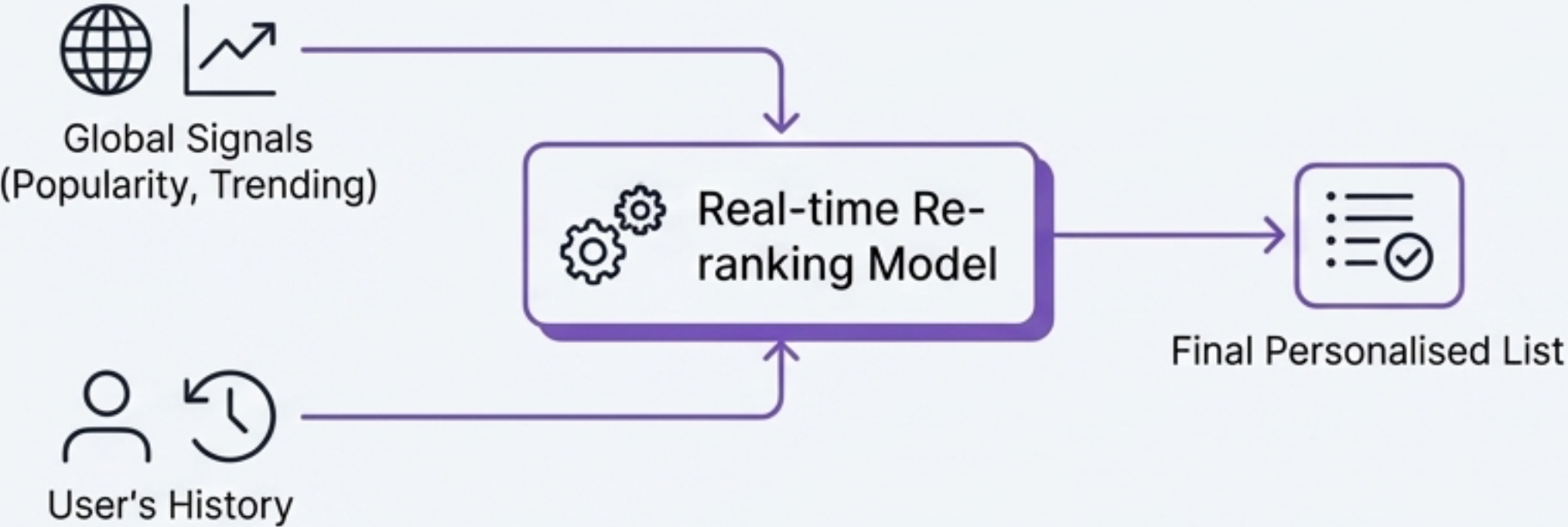
The Flink job consumes events directly from Kafka in real-time.

It uses stateful processing and tumbling windows (e.g., 5-minute windows) to calculate event **velocity**—the acceleration of engagement for a product compared to its baseline.

This approach detects items that are “catching fire” right now, a different signal from slow-moving historical popularity.

Key Insight: Flink’s stateful, event-time processing allows us to detect trending items with an end-to-end latency of under 5 minutes, meeting our NFR for data freshness.

The 'For You' Engine: Crafting Personalised Rankings



Core Strategy:

Personalisation is primarily a real-time **re-ranking layer** applied to the candidate set generated from global signals. This is supplemented by offline Collaborative Filtering models computed with Spark.

Handling the Cold Start Problem

User Type	Signal Availability	Strategy
Anonymous	None	Global popularity ranking
Anonymous + Session	Current session only	Boost categories from session views
Low Activity (<10 views)	Limited history	Blend: 70% global + 30% personal
Active User (>50 views)	Rich history	Blend: 30% global + 70% personal

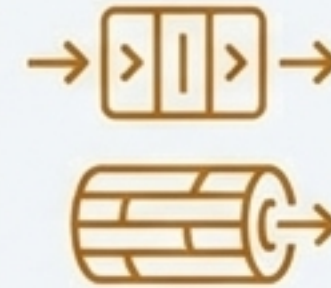
The Right Tool for the Job: A Purpose-Driven Tech Stack

Data Stores



PostgreSQL + Citus: Chosen for ACID compliance for core product data, with the Citus extension enabling horizontal sharding for future scale.

Messaging & Ingestion



Apache Kafka: Selected for its durable, replayable log architecture, which guarantees event delivery and supports multiple, independent consumer pipelines.

Real-Time Processing



Apache Flink: Preferred over other stream processors for its true event-time processing and exactly-once state guarantees, which are crucial for accurate trend detection.

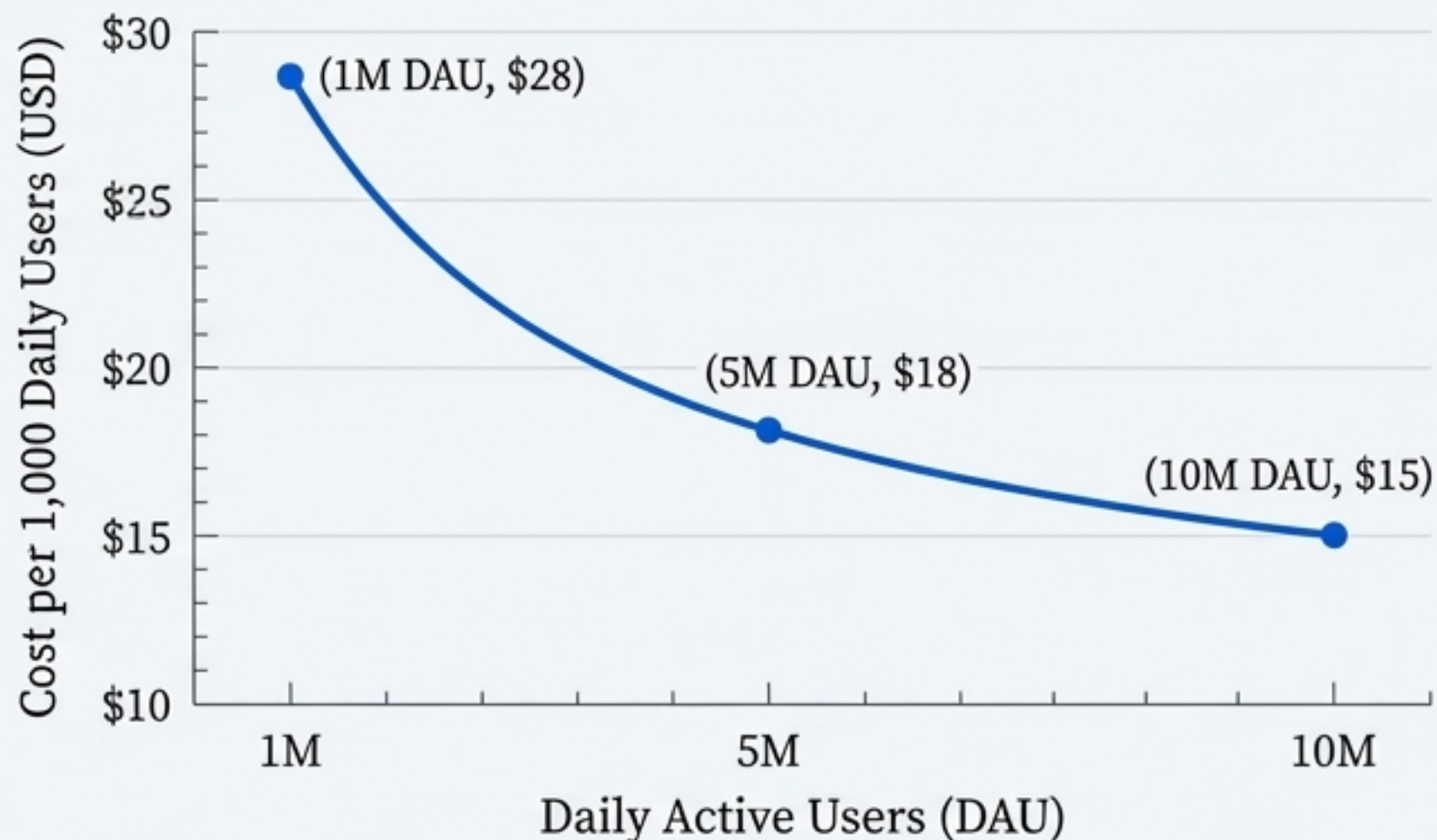
Orchestration & Infrastructure



Kubernetes: Used as the foundation for its declarative deployments, self-healing capabilities, and robust auto-scaling (HPA), ensuring operational efficiency.

From 1 Million to 10 Million Users: Engineering for Growth

Economies of Scale



Key Scalability Strategies



Stateless Services: All API services are stateless and run on Kubernetes, enabling horizontal scaling via HPA.



Database Scaling: PostgreSQL scales via read replicas for read-heavy workloads and Citus for sharding when write throughput becomes a bottleneck.



Decoupled Processing: The Kafka, Flink, and Spark clusters scale independently based on their specific processing loads.

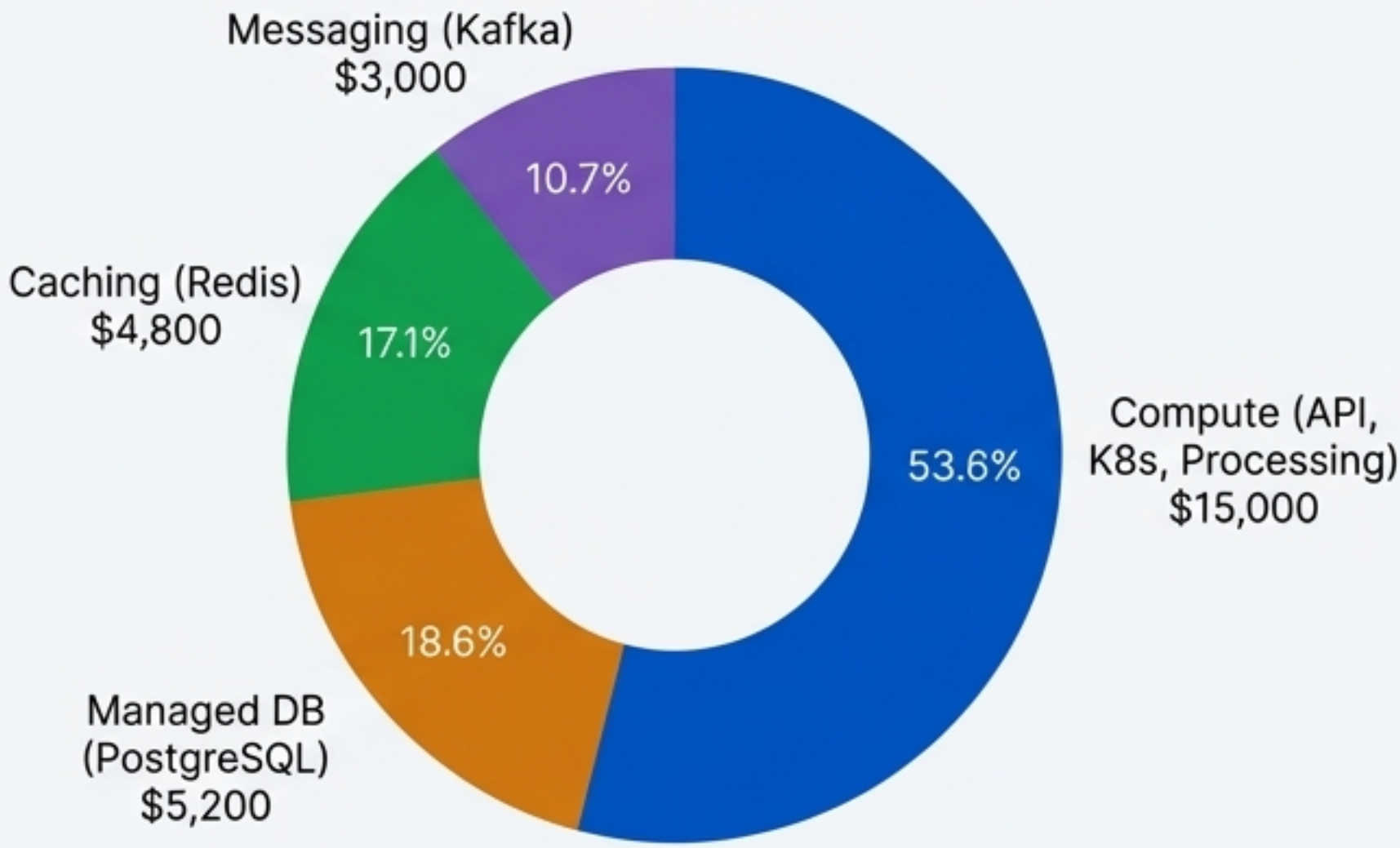
The architecture is designed to scale horizontally, ensuring that adding capacity for the next 10M users does not require a fundamental re-architecture.

Planning for Failure: A System That Heals and Degrades Gracefully

Failure Scenarios			Graceful Degradation Path	
Failure Scenario	Mitigation	Recovery Time		
 Redis Node Down	Auto-failover via Sentinel, fallback to PostgreSQL	< 30s	 Normal: Full experience with caching, real-time trends, and personalisation.	
 PostgreSQL Primary Down	Automatic promotion of replica to new primary	< 60s	 Degraded (No Cache): Latency increases as requests fall back to PostgreSQL.	
 Flink Job Crash	Automatic restart from last checkpoint, replay from Kafka	< 5 min	 Degraded (No Personalisation): Personalisation service fails; serve global popularity only.	
The system is designed to degrade gracefully, prioritising availability and a baseline user experience over feature-completeness during outages.			 Unavailable: Return 503 with Retry-After header.	

The Economics of Discovery: Cost Profile for 1M DAU

Monthly Cost Breakdown (1M DAU)



Top Cost Optimisation Strategies

-  1. **Reserved Instances:** Commit to 1-year terms for baseline compute needs to achieve 30-40% savings.
-  2. **Spot Instances:** Leverage for fault-tolerant batch workloads like Spark jobs for up to 70% savings.
-  3. **Aggressive Auto-scaling:** Scale down services, including processing clusters, during off-peak hours.

Total Estimated Monthly Cost: ~\$28,000

Cost per 1,000 Daily Active Users: ~\$28

The Blueprint to Reality: A 20-Week Phased Implementation



Foundation

Weeks 1-4: Deploy K8s, PostgreSQL, Redis. Implement the basic Browse API and core monitoring.



Batch Pipeline

Weeks 5-8: Deploy Kafka. Implement event collection and the Spark popularity pipeline scheduled via Airflow.



Real-Time Pipeline

Weeks 9-12: Deploy Flink. Implement the 'hot items' streaming job and integrate it into the API.



Personalisation

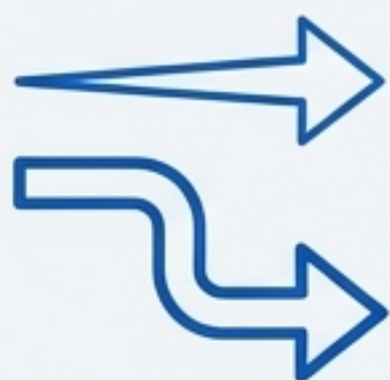
Weeks 13-16: Implement the re-ranking service, feature store integration, and A/B testing framework.



Production Hardening

Weeks 17-20: Conduct load testing, chaos engineering, security audits, and prepare for launch.

Our Guiding Architectural Principles



Separate Read and Write Paths

Allows independent optimisation and scaling of serving latency from data ingestion throughput.



Use the Right Data Store for the Job

Employ polyglot persistence—PostgreSQL for consistency, Redis for speed, a Data Lake for analytics.



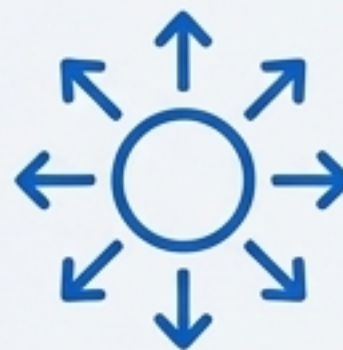
Use the Right Processing for the Right Speed

Leverage batch (Spark) for deep historical analysis and streaming (Flink) for immediate insights.



Design for Failure, Not Against It

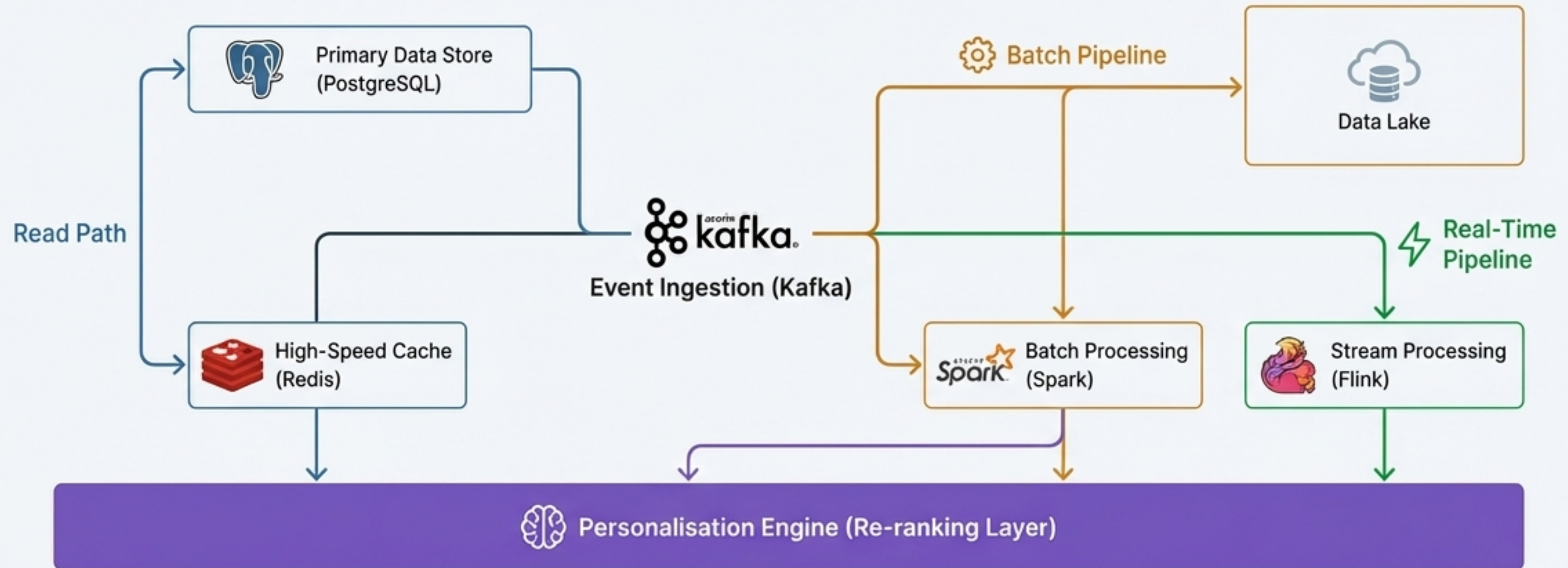
Build in resilience, automated recovery, and graceful degradation from the start.



Decouple Everything with Events

Use a distributed log like Kafka as the central nervous system to create a flexible, evolvable architecture.

A Complete Blueprint for Scalable Product Discovery



This architecture provides a robust, scalable, and cost-efficient foundation for delivering a world-class e-commerce browsing experience. It balances real-time responsiveness with deep analytical insight, creating a system that is built to last and designed to evolve.



Scan for the complete system design document.